NBA Analysis Project

Kailot C. Harris

2026-09-01

Introduction

```
#install.packages(c( "tidymodels", "hoopR", "slider"))
#install.packages("ranger")
#install.packages("vip", type = "source")

#install.packages(c("dplyr", "purrr", "rlang"), type = "binary")

library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
## Warning: package 'purrr' was built under R version 4.5.3
```

```
## Warning: package 'dplyr' was built under R version 4.5.3
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.5.3
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.5.0 --
## v broom        1.0.12     v rsample      1.3.2
## v dials        1.4.4      v tailor       0.1.0
## v infer        1.1.0      v tune         2.1.0
## v modeldata    1.6.0      v workflows    1.3.0
## v parsnip      1.6.0      v workflowsets 1.1.1
## v recipes      1.4.0      v yardstick    1.4.0
```

```
## Warning: package 'dials' was built under R version 4.5.3
```

```
## Warning: package 'infer' was built under R version 4.5.3
```

```
## Warning: package 'modeldata' was built under R version 4.5.3
```

```
## Warning: package 'parsnip' was built under R version 4.5.3
```

```
## Warning: package 'recipes' was built under R version 4.5.3
```

```
## Warning: package 'rsample' was built under R version 4.5.3
## Warning: package 'tailor' was built under R version 4.5.3
## Warning: package 'tune' was built under R version 4.5.3
## Warning: package 'workflows' was built under R version 4.5.3
## Warning: package 'workflowsets' was built under R version 4.5.3
## Warning: package 'yardstick' was built under R version 4.5.3
## -- Conflicts --------------------------------------- tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
```

```
library(hoopR)
```

```
## Warning: package 'hoopR' was built under R version 4.5.3
```

```
library(slider)
```

```
## Warning: package 'slider' was built under R version 4.5.3
```

Load the data from hoopr api

```
box_scores <- hoopR::load_nba_team_box(seasons = 2019:2026)

#glimpse(box_scores)
```

calculate features using box_scores data set features include: effective field goal percentage, home game advantage, last game outcome, turnover rate, free throw rate, and true shooting percentage. Other parameters are also included as features and most values in the feature set are calculated on a rolling average.

```
# Calculate rolling averages of Four Factors (e.g., eFG%, Turnover Rate, etc.)
full_features_df <- box_scores |>
  arrange(team_id, game_date) |>
  group_by(team_id) |>
  mutate(
    #define home-game advantage
    hga = if_else(str_detect(team_home_away, "away"),0,1),
    #define last game outcome
    lgo = lag(as.integer(team_winner), n = 1, default = NA),
    #define effective field goal percentage
    efg = (field_goals_made + 0.5*three_point_field_goals_made)/(field_goals_attempted),
    #define turnover rate percentage
    tov = (total_turnovers)/(field_goals_attempted + 0.44 * free_throws_attempted + total_turnovers),
    #define free throw rate
    ftr = free_throws_attempted/field_goals_attempted,
    #define true shooting percentage
    ts = team_score / (2* (field_goals_attempted + (0.44 * free_throws_attempted))),
    # Rolling 10-game average of offensive rating and shooting %
    roll_pts_scored = slide_dbl(team_score, mean, .before = 10, .after = -1),
    roll_field_goal_pct = slide_dbl(field_goal_pct, mean, .before = 10, .after = -1),
    roll_oreb = slide_dbl(offensive_rebounds, mean, .before = 10, .after = -1),
    roll_dreb = slide_dbl(defensive_rebounds, mean, .before = 10, .after = -1),
```

```
    roll_reb = slide_dbl(total_rebounds, mean, .before = 10, .after = -1),
    roll_ast = slide_dbl(assists, mean, .before = 10, .after = -1),
    roll_stl = slide_dbl(steals, mean, .before = 10, .after = -1),
    roll_blk = slide_dbl(blocks, mean, .before = 10, .after = -1),
    roll_tov = slide_dbl(turnovers, mean, .before = 10, .after = -1),
    roll_efg = slide_dbl(efg, mean, .before = 10, .after = -1),
    roll_tovp = slide_dbl(tov, mean, .before = 10, .after = -1),
    roll_ftr = slide_dbl(ftr, mean, .before = 10, .after = -1),
    roll_ts = slide_dbl(ts, mean, .before = 10, .after = -1),

    days_rest = as.numeric(game_date - lag(game_date))
  ) |>
  ungroup()

#glimpse(features_df)
```

Select only the rows necessary for feature analysis.

```
features_df <- full_features_df |>
  select(game_id, season, game_date, team_id, team_winner, team_home_away, opponent_team_id, roll_pts_s

glimpse(features_df)
```

```
## Rows: 20,480
## Columns: 21
## $ game_id          <int> 401070697, 401070705, 401070718, 401070733, 401070757~
## $ season           <int> 2019, 2019, 2019, 2019, 2019, 2019, 2019, 2019, 2019,~
## $ game_date        <date> 2018-10-17, 2018-10-19, 2018-10-21, 2018-10-24, 2018~
## $ team_id          <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ team_winner      <lgl> FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE,~
## $ team_home_away   <chr> "away", "away", "away", "home", "home", "away", "away~
## $ opponent_team_id <int> 18, 29, 5, 6, 4, 20, 5, 23, 14, 30, 18, 8, 13, 9, 7, ~
## $ roll_pts_scored  <dbl> NaN, 107.0000, 112.0000, 119.0000, 117.0000, 110.6000~
## $ roll_oreb        <dbl> NaN, 7.000000, 9.000000, 9.000000, 9.250000, 9.600000~
## $ roll_dreb        <dbl> NaN, 38.00000, 33.50000, 34.00000, 36.50000, 36.60000~
## $ roll_ast         <dbl> NaN, 19.00000, 23.00000, 26.00000, 26.50000, 25.20000~
## $ roll_stl         <dbl> NaN, 7.000000, 8.000000, 8.333333, 7.750000, 8.000000~
## $ roll_blk         <dbl> NaN, 3.000000, 4.000000, 5.000000, 5.250000, 5.800000~
## $ roll_tov         <dbl> NaN, 24.00000, 21.50000, 18.33333, 17.50000, 18.40000~
## $ roll_efg         <dbl> NaN, 0.5111111, 0.5447122, 0.5638217, 0.5451190, 0.51~
## $ roll_tovp        <dbl> NaN, 0.1968504, 0.1790430, 0.1531572, 0.1462119, 0.15~
## $ roll_ftr         <dbl> NaN, 0.2000000, 0.3168675, 0.2758708, 0.2920680, 0.28~
## $ roll_ts          <dbl> NaN, 0.5463644, 0.5691150, 0.5878217, 0.5734637, 0.54~
## $ hga              <dbl> 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1,~
## $ lgo              <int> NA, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0~
## $ days_rest        <dbl> NA, 2, 2, 3, 3, 2, 1, 2, 2, 3, 1, 2, 2, 2, 2, 2, 2, 2~
```

Now that the features are sorted, let's divide up the data into training and validation sets. But first, let's disentangle the data slightly. Let's create two datasets: one for home team performance and one for away team performance.

```
home_df <- features_df |>
  filter(team_home_away=="home") |>
  select(game_id, season, game_date,
         home_team = team_id,
```

```
        home_win = team_winner,
        home_roll_pts = roll_pts_scored,
        home_roll_oreb = roll_oreb,
        home_roll_dreb = roll_dreb,
        home_roll_ast = roll_ast,
        home_roll_stl = roll_stl,
        home_roll_blk = roll_blk,
        home_roll_tov = roll_tov,
        home_roll_efg = roll_efg,
        home_roll_tovp = roll_tovp,
        home_roll_ftr = roll_ftr,
        home_roll_ts = roll_ts,
        home_days_rest = days_rest,
  )

away_df <- features_df |>
  filter(team_home_away=="away") |>
  select(game_id,
        away_team = team_id,
        away_win = team_winner,
        away_roll_pts = roll_pts_scored,
        away_roll_oreb = roll_oreb,
        away_roll_dreb = roll_dreb,
        away_roll_ast = roll_ast,
        away_roll_stl = roll_stl,
        away_roll_blk = roll_blk,
        away_roll_tov = roll_tov,
        away_roll_efg = roll_efg,
        away_roll_tovp = roll_tovp,
        away_roll_ftr = roll_ftr,
        away_roll_ts = roll_ts,
        away_days_rest = days_rest,
  )
#merge home and away data frames with inner join on `game_id`
model_df <- inner_join(home_df, away_df, by = "game_id") |>
  mutate(
    target_home_win = factor(if_else(home_win==TRUE, "Win", "Loss")),
    diff_roll_pts  = home_roll_pts - away_roll_pts,
    diff_roll_oreb  = home_roll_oreb - away_roll_oreb,
    diff_roll_dreb  = home_roll_dreb - away_roll_dreb,
    diff_rol_treb = (home_roll_oreb + home_roll_dreb)-(away_roll_oreb + away_roll_dreb),
    diff_roll_ast  = home_roll_ast - away_roll_ast,
    diff_roll_stl  = home_roll_stl - away_roll_stl,
    diff_roll_blk  = home_roll_blk - away_roll_blk,
    diff_roll_tov  = home_roll_tov - away_roll_tov,
    diff_roll_efg  = home_roll_efg - away_roll_efg,
    diff_roll_tovp  = home_roll_tovp - away_roll_tovp,
    diff_roll_ftr  = home_roll_ftr - away_roll_ftr,
    diff_roll_ts  = home_roll_ts - away_roll_ts,
    diff_days_rest = home_days_rest - away_days_rest,

  )
```

Model training

```
train_data <- model_df |>
  filter(season<2026)

test_data <- model_df |>
  filter(season>=2026)

nba_recipe <- recipe(target_home_win ~ ., data = train_data) |>
  step_rm(game_id, game_date, home_team, away_team, home_win, away_win, home_roll_ts, home_roll_ftr, hor
  step_normalize(all_numeric_predictors())

rf_spec <- rand_forest(trees=500, mode = "classification") |>
  set_engine("ranger", importance = "impurity")

nba_workflow <- workflow() |>
  add_recipe(nba_recipe) |>
  add_model(rf_spec)

nba_fit <- fit(nba_workflow, data = train_data)
```

Model predictions

Predict on the 2026 test season

```
predictions <- predict(nba_fit, test_data) |>
  bind_cols(predict(nba_fit, test_data, type = "prob")) |>
  bind_cols(test_data)

# Metrics (Accuracy, ROC-AUC)
predictions |>
  metrics(truth = target_home_win, estimate = .pred_class)
```

```
## # A tibble: 2 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.604
## 2 kap      binary         0.179
```

```
predictions |>
  roc_auc(truth = target_home_win, .pred_Win, event_level = "second")
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.647
```

Plots

Check the feature importance

```
# 1. Extract the ranger engine directly
rf_engine <- extract_fit_engine(nba_fit)

# 2. Extract and sort purely in Base R (no dplyr needed)
imp_vec <- rf_engine$variable.importance
```

```r
importance_df <- data.frame(
  feature = names(imp_vec),
  importance = as.numeric(imp_vec)
)

# Sort descending and take top 10
importance_df <- importance_df[order(-importance_df$importance), ]
top10_df <- head(importance_df, 10)

# 3. Plot with ggplot2
ggplot(top10_df, aes(x = reorder(feature, importance), y = importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Top Features for Predicting NBA Winners",
    x = "Feature",
    y = "Importance (Impurity Gini)"
  )
```

